

EXPERMENT: Alpha & Beta pruning algorithm for gaming

Aim

To implement the Alpha-Beta Pruning algorithm in Python for gaming and efficiently determine the best move by eliminating unnecessary branches of the game tree.

Objectives

1. To understand the Alpha-Beta Pruning technique.
2. To improve the efficiency of the Minimax algorithm.
3. To identify unnecessary game-tree branches.
4. To select the optimal move for a player.
5. To reduce the number of nodes evaluated during game search.

Algorithm

1. Start with the root node of the game tree.
2. Initialize $\alpha = -\infty$ and $\beta = +\infty$.
3. For a MAX node, select the maximum value.
4. Update α with the maximum value found.
5. For a MIN node, select the minimum value.
6. Update β with the minimum value found.
7. If $\beta \leq \alpha$, prune the remaining branches.
8. Continue until the terminal nodes are reached.
9. Return the optimal value to the root.
10. Select the move corresponding to the optimal value.

Code

```
def alpha_beta(depth, node, maximizing, alpha, beta, tree):  
    if depth == 2:  
        return tree[node]  
    if maximizing:  
        best = -float('inf')  
        for child in [node * 2, node * 2 + 1]:  
            value = alpha_beta(  
                depth + 1, child, False, alpha, beta, tree  
            )
```

```

        best = max(best, value)

        alpha = max(alpha, best)

        if beta <= alpha:
            print("Branch pruned at node", child)
            break

    return best

else:
    best = float('inf')

    for child in [node * 2, node * 2 + 1]:
        value = alpha_beta(
            depth + 1, child, True, alpha, beta, tree
        )

        best = min(best, value)

        beta = min(beta, best)

        if beta <= alpha:
            print("Branch pruned at node", child)
            break

    return best

tree = {
    4: 3,
    5: 5,
    6: 2,
    7: 9
}

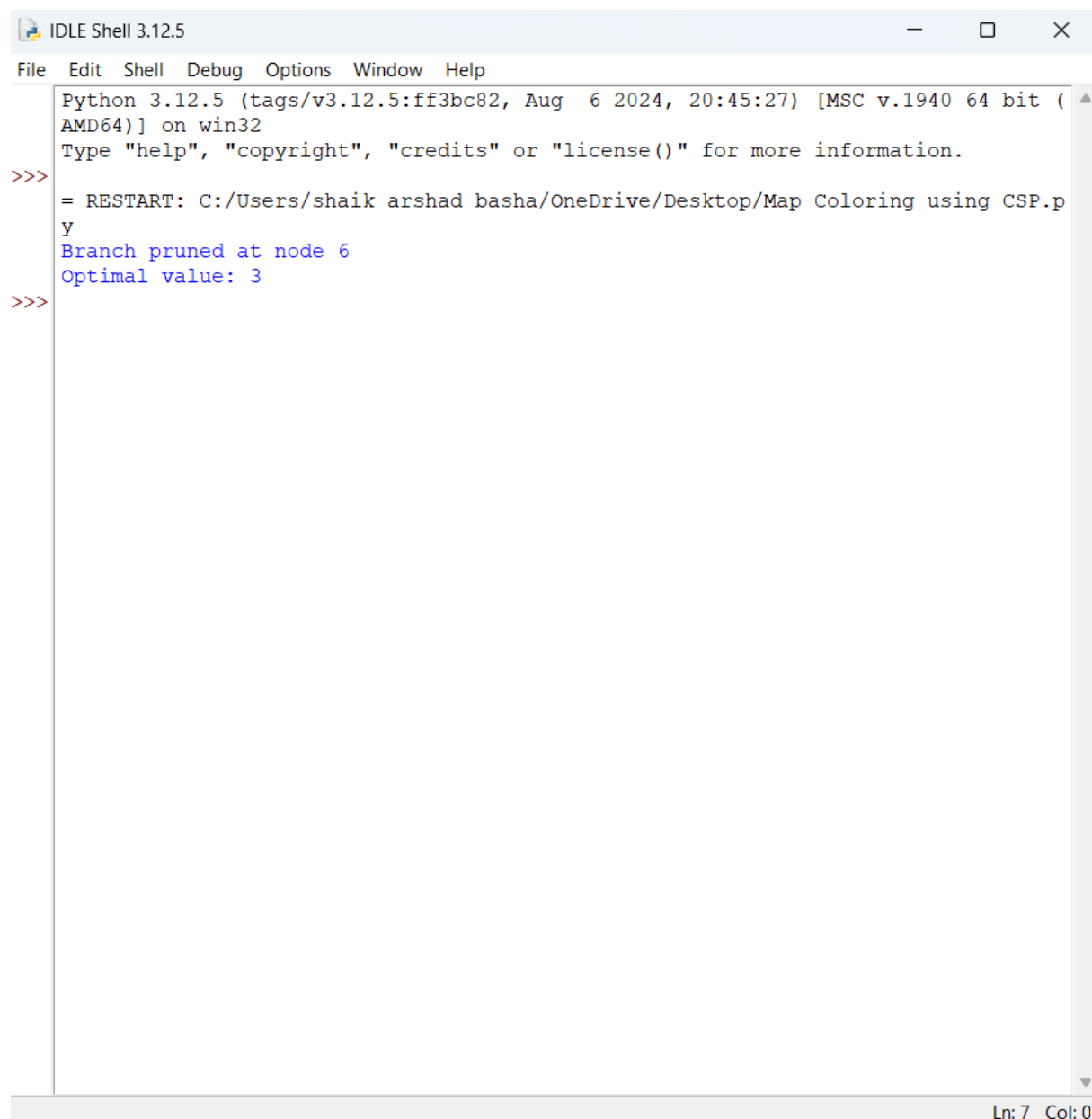
alpha = -float('inf')
beta = float('inf')

result = alpha_beta(0, 1, True, alpha, beta, tree)

print("Optimal value:", result)

```

Output

A screenshot of an IDLE Shell 3.12.5 window. The window has a title bar with the text 'IDLE Shell 3.12.5' and standard window controls (minimize, maximize, close). Below the title bar is a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area shows the following output: 'Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32', 'Type "help", "copyright", "credits" or "license()" for more information.', a prompt '>>>', a line '= RESTART: C:/Users/shaik arshad basha/OneDrive/Desktop/Map Coloring using CSP.py', 'Branch pruned at node 6', 'Optimal value: 3', and another prompt '>>>'. The status bar at the bottom right shows 'Ln: 7 Col: 0'.

Result

The Alpha-Beta Pruning algorithm was successfully implemented. It obtained the optimal value while pruning branches that could not affect the final decision.

Conclusion

Alpha-Beta Pruning improves the efficiency of Minimax by eliminating unnecessary branches of the game tree. Therefore, it allows AI game players to search deeper levels of the game tree with fewer computations.